

MATLAB[®] **para ingenieros**

Holly Moore



Entrada y salida controladas por el usuario

Objetivos

Después de leer este capítulo, el alumno será capaz de

- indicar al usuario que ingrese entrada a un programa de archivo-m.
- crear salida con la función **disp**.
- crear salida formateada con **fprintf**.
- usar técnicas gráficas para proporcionar entrada al programa.
- usar el modo cell para modificar y correr programas de archivo-m.

INTRODUCCIÓN

Hasta el momento, MATLAB se ha usado de dos maneras: como una memoria de trabajo auxiliar (scratch pad) en la ventana de comandos y para escribir programas simples (archivos-m script y funciones) en la ventana de edición. En ambos casos se supuso que el programador era el usuario. En este capítulo se avanzará hacia los programas más complicados, escritos en la ventana de edición, suponiendo que el programador y el usuario pueden ser personas diferentes. Esto hará necesario comunicarse con el usuario a través de comandos de entrada y salida, en lugar de reescribir el código real para resolver problemas similares. MATLAB ofrece funciones internas para permitir al usuario comunicarse con un programa conforme se ejecuta. El comando **input** detiene el programa y conmina al usuario a ingresar una entrada; los comandos **disp** y **fprintf** proporcionan salida a la ventana de comandos.

7.1 ENTRADA DEFINIDA POR EL USUARIO

Aunque se han escrito programas en archivos-m script, se supuso que el programador (usted) y el usuario eran la misma persona. Para correr el programa con diferentes valores de entrada, en realidad se cambió parte del código. Se pueden recrear programas más generales al permitir al usuario ingresar valores de una matriz desde el teclado mientras el programa corre. La función **input** le permite hacer esto. Despliega una cadena de texto en la ventana de comando y luego espera que el usuario proporcione la entrada solicitada. Por ejemplo,

```
z = input('Ingresa un valor')
```

despliega

Ingresa un valor

en la ventana de comandos. Si el usuario ingresa un valor como

el programa asigna el valor 5 a la variable **z**. Si el comando **input** no termina con un punto y coma, el valor ingresado se despliega en la pantalla:

```
z =  
5
```

El mismo enfoque se puede usar para ingresar una matriz uni o bidimensional. El usuario debe proporcionar los paréntesis y delimitadores adecuados (comas y puntos y coma). Por ejemplo,

```
z = input('Ingrese valores para z entre corchetes')
```

Idea clave: la función **input** se puede usar para comunicarse con el usuario del programa.

pide al usuario que ingrese una matriz como

```
[ 1, 2, 3; 4, 5, 6]
```

y responde con

```
z =  
1 2 3  
4 5 6
```

Este valor de entrada de **z** se puede usar entonces en cálculos subsecuentes por el archivo-m script.

Los datos ingresados con **input** no necesitan ser información numérica. Suponga que se conmina al usuario con el comando

```
x=input('Ingrese su nombre en apóstrofes')
```

e ingrese

```
'Holly'
```

cuando se le indique. Puesto que no se usó punto y coma al final del comando **input**, MATLAB responderá

```
x =  
Holly
```

Note en la ventana del área de trabajo que **x** se menciona como un arreglo carácter de 1×5 :

Nombre	Valor	Tamaño	Bytes	Clase
abc x	'Holly'	1×5	6	char

Si ingresa una cadena (en MATLAB, las cadenas son arreglos carácter), debe encerrar los caracteres en apóstrofes. Sin embargo, una forma alternativa del comando **input** alerta a la función a esperar entrada carácter sin los apóstrofes, al especificar entrada cadena en el segundo campo:

```
x=input('Ingrese su nombre', 's')
```

Ahora sólo necesita ingresar los caracteres, como

```
Ra1ph
```

y el programa responde con

```
x =  
Ra1ph
```

Ejercicio de práctica 7.1

1. Cree un archivo-m para calcular el área A de un triángulo:

$$A = \frac{1}{2} \text{ base altura}$$

Conmíne al usuario a ingresar los valores para la base y la altura.

2. Cree un archivo-m para encontrar el volumen V de un cilindro circular recto:

$$V = \pi r^2 h$$

Conmíne al usuario a ingresar los valores de r y h .

3. Cree un vector desde 1 hasta n , y permita al usuario ingresar el valor de n .
4. Cree un vector que comience en a , termine en b y tenga un espaciado de c . Permita al usuario ingresar todos estos parámetros.

Objetos en caída libre

Analice el comportamiento de un objeto en caída libre. (Véase la figura 7.1.) La ecuación relevante es

$$d = \frac{1}{2} g t^2$$

donde d = distancia que recorre el objeto,
 g = aceleración debida a la gravedad, y
 t = tiempo en que el objeto recorre la distancia d .

Se debe permitir al usuario especificar el valor de g , la aceleración debida a la gravedad, y un vector de valores de tiempo.

1. Establezca el problema.
Encontrar la distancia que recorre un objeto en caída libre y graficar los resultados.
2. Describir las entradas y salidas.

Entrada Valor de g , la aceleración debida a la gravedad, proporcionada por el usuario
 Tiempo, proporcionado por el usuario

Salida Distancias
 Gráfica de distancia contra tiempo

3. Desarrolle un ejemplo a mano.

$$d = \frac{1}{2} g t^2, \text{ de modo que, en la luna, a 100 segundos,}$$

$$d = \frac{1}{2} \times 1.6 \text{ m/s}^2 \times 100^2 \text{ s}^2$$

$$d = 8000 \text{ m}$$

4. Desarrolle una solución MATLAB.

```
%Example 7.1
%Free fall
```

EJEMPLO 7.1

Figura 7.1
 Torre inclinada de Pisa.
 (Cortesía de Tim Galligan.)

```

clear, clc
%Solicite entrada del usuario
g = input('¿Cuál es el valor de aceleración debida a gravedad?')
start=input('¿Qué tiempo de inicio le gustaría? ')
finish = input('¿Qué tiempo final le gustaría? ')
incr = input('¿Qué incrementos de tiempo le gustaría calcular? ')
t=start:incr:finish;
%Calcula la distancia
d=1/2*g*t.^2;
%Grafica los resultados
loglog(t,d)
title('Distancia recorrida en caída libre')
xlabel('tiempo, s'),ylabel('distancia, m')
%Encuentra la distancia máxima recorrida
final_distance = max(d)

```

La interacción en la ventana de comandos es la siguiente:

```

¿Cuál es el valor de aceleración debida a gravedad? 1.6
g =
    1.6000
¿Qué tiempo de inicio le gustaría? 0
start =
    0
¿Qué tiempo final le gustaría? 100
finish =
    100
¿Qué incrementos de tiempo le gustaría calcular? 10
incr =
    10
final_distance =
    8000

```

En la figura 7.2 se grafican los resultados del ejemplo 7.1.

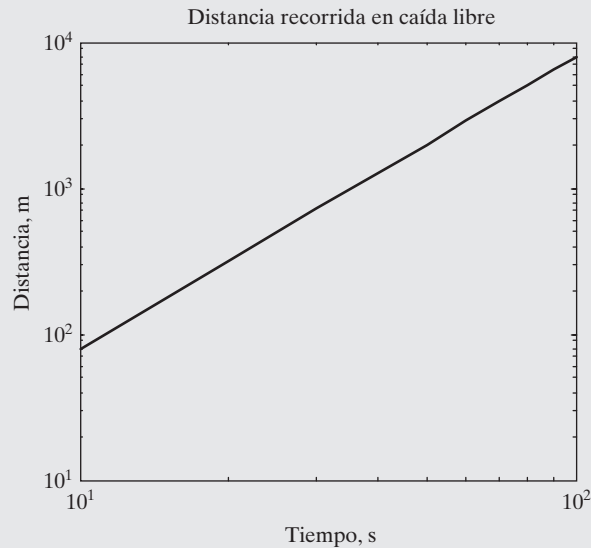


Figura 7.2

Distancia recorrida cuando la aceleración es 1.6 m/s^2 .

5. Ponga a prueba la solución.

Compare la solución MATLAB con la solución a mano. Dado que el usuario puede controlar la entrada, se ingresan los datos usados en la solución a mano. MATLAB dice que la distancia final recorrida es 8000 m, la cual corresponde a la distancia recorrida luego de 100 segundos, puesto que se ingresó 100 segundos como el tiempo final.

7.2 OPCIONES DE SALIDA

Existen muchas formas de desplegar los contenidos de una matriz. La más simple es ingresar el nombre de la matriz, sin punto y coma. El nombre de la matriz se repetirá y los valores de la matriz se desplegarán comenzando en la línea siguiente. Por ejemplo, defina primero una matriz **x**:

```
x = 1:5;
```

Puesto que hay punto y coma al final del enunciado de asignación, los valores en **x** no se repiten en la ventana de comandos. Sin embargo, si quiere desplegar **x** más tarde en su programa, simplemente escriba en el nombre de variable

```
x
```

lo que regresa

```
x =
     1     2     3     4     5
```

MATLAB ofrece otros dos enfoques para desplegar resultados: la función **disp** y la función **fprintf**.

Idea clave: la función **disp** puede desplegar arreglos carácter o arreglos numéricos.

7.2.1 Función despliegue (display)

La función despliegue (**display**, **disp**) se puede usar para desplegar los contenidos de una matriz sin imprimir el nombre de matriz. Por lo tanto,

```
disp(x)
```

regresa

```
     1     2     3     4     5
```

El comando **display** también se puede usar para desplegar una cadena (texto encerrado en marcas de comilla simple o apóstrofe). Por ejemplo,

```
disp('Los valores en la matriz x son:');
```

regresa

```
Los valores en la matriz x son:
```

Cuando ingresa una cadena como entrada en la función **disp**, en realidad ingresa un arreglo de información carácter. Intente ingresar lo siguiente en la línea de comando:

```
'Los valores en la matriz x son:'
```

arreglo carácter:
almacena información
carácter

MATLAB responde

```
ans =  
'Los valores en la matriz x son:'
```

La ventana del área de trabajo cita **ans** como un arreglo carácter 1×32 .

Nombre	Tamaño	Bytes	Clase
abc ans	1×32	90	char array

Los arreglos carácter almacenan información carácter en arreglos similar a los arreglos numéricos. Los caracteres pueden ser letras, números, puntuación e incluso algunos caracteres no desplegables. Cada carácter, incluidos espacios, es un elemento en el arreglo carácter. Cuando se ejecutan las dos funciones de despliegue

```
disp('Los valores en la matriz x son:');  
disp(x)
```

Idea clave: los
caracteres pueden
ser letras, números o
símbolos.

MATLAB responde

```
Los valores en la matriz x son:  
1 2 3 4 5
```

Note que las dos funciones **disp** se despliegan en líneas separadas. Puede evitar esta característica al crear una matriz combinada de sus dos salidas, con el uso de la función **num2str** (número a cadena). El proceso se llama concatenación y crea un solo arreglo carácter. Por ende,

```
disp(['Los valores en el arreglo x son: ' num2str(x)])
```

regresa

```
Los valores en el arreglo x son: 1 2 3 4 5
```

La función **num2str** cambia un arreglo de números en un arreglo de caracteres. En el ejemplo anterior se usó **num2str** para transformar la matriz **x** a un arreglo carácter, que entonces se combinó con la primera cadena (mediante corchetes []) para hacer un arreglo carácter más grande. Puede ver la matriz resultante al escribir

```
A = ['Los valores en la matriz x son: ' num2str(x)]
```

que regresa

```
A =  
Los valores en el arreglo x son: 1 2 3 4 5
```

Al verificar en la ventana del área de trabajo, se ve que **A** es una matriz 1×45 . La ventana del área de trabajo también indica que la matriz contiene datos carácter en lugar de información numérica. Esto se evidencia tanto por el icono en frente de **A** como en la columna class.

Nombre	Tamaño	Bytes	Clase
abc A	1×45	90	char array

Sugerencia

Si quiere incluir un apóstrofe en una cadena, necesita ingresar el apóstrofe dos veces. Si no lo hace, MATLAB interpretará el apóstrofe como terminación de la cadena. Un ejemplo del uso de dos apóstrofes es

```
disp('The moon's gravity is 1/6th that of the earth')
```

Puede usar una combinación de funciones **input** y **disp** para imitar una conversación. Intente crear y correr el siguiente archivo-m:

```
disp('Hi There');
disp('I'm your MATLAB program');
name=input('Who are you?','s');
disp(['Hi ',name]);
answer=input('Don't you just love computers?', 's');
disp([answer, '?']);
disp('Computers are very useful');
disp('You'll use them a lot in college!!');
disp('Good luck with your studies')
pause(2);
disp('Bye bye')
```

7.2.2 Salida formateada

La función **fprintf** (función impresión formateada) le da incluso mayor control sobre la salida de la que tiene con la función **disp**. Además de desplegar valores tanto texto como matriz, puede especificar el formato a usar al desplegar los valores, y puede especificar cuándo saltar a una nueva línea. Si usted es programador C, estará familiarizado con la sintaxis de esta función. Con pocas excepciones, la función MATLAB **fprintf** usa las mismas especificaciones de formato que la función **fprintf** C. Esto difícilmente es sorprendente, pues MATLAB se escribió en C. (MATLAB originalmente se escribió en FORTRAN y más tarde se reescribió en C.)

La forma general del comando **fprintf** contiene dos argumentos, uno cadena y otro una lista de matrices:

```
fprintf(format-string, var,...)
```

Considere el siguiente ejemplo:

```
cows = 5;
fprintf('Hay %f vacas en el pastizal', vacas)
```

La cadena, que es el primer argumento dentro de la función **fprintf**, contiene un marcador de posición (placeholder) (%) donde se insertará el valor de la variable (en este caso, **vacas**). El marcador de posición también contiene información de formato. En este ejemplo, **%f** le pide a MATLAB desplegar el valor de **vacas** en un formato de punto fijo por defecto. El formato por defecto despliega seis lugares después del punto decimal:

```
Hay 5.000000 vacas en el pastizal
```

Además del formato de punto fijo por defecto, MATLAB le permite especificar un formato exponencial, **%e**, o le permite a MATLAB elegir el que sea

Tabla 7.1 Formato de tipo de campo

Tipo de campo	Resultado
%f	notación punto fijo o decimal
%e	notación exponencial
%g	la que sea más corta, %f o %e
%c	información carácter
%s	cadena de caracteres

En la característica help se describen tipos de campos adicionales.

Idea clave: la función `fprintf` le permite controlar cómo se despliegan los números.

más corto, punto fijo o exponencial (%g). También le permite desplegar información carácter (%c) o una cadena de caracteres (%s). La tabla 7.1 ilustra varios formatos.

MATLAB no comienza automáticamente una nueva línea después de ejecutar una función `fprintf`. Si usted intentó el ejemplo de comando `fprintf` anterior, probablemente notó que el prompt del comando está en la misma línea que la salida:

```
Hay 5.000000 vacas en el pastizal >>
```

Si ejecuta otro comando, los resultados aparecerán en la misma línea en lugar de moverlos abajo. Por tanto, si se emiten los nuevos comandos

```
cow = 6;  
fprintf('Hay %f vacas en el pastizal', vacas);
```

desde un archivo-m, MATLAB continúa el despliegue de ventana de comandos en la misma línea:

```
Hay 5.000000 vacas en el pastizal Hay 6.000000  
vacas en el pastizal
```

Idea clave: la función `fprintf` le permite desplegar tanto caracteres como información numérica con un solo comando.

Para hacer que MATLAB inicie una nueva línea, necesitará usar `\n`, llamado salto de línea (linefeed), al final de la cadena. Por ejemplo, el código

```
cows=5;  
fprintf('Hay %f vacas en el pastizal \n', vacas)  
cows = 6;  
fprintf('Hay %f vacas en el pastizal \n', vacas)
```

regresa la siguiente salida:

```
Hay 5.000000 vacas en el pastizal  
Hay 6.000000 vacas en el pastizal
```

Sugerencia

La diagonal inversa (\) y la diagonal normal (/) son caracteres diferentes. Es un error común confundirlas, ¡y luego el comando linefeed no funciona! En vez de ello, la salida en la ventana de comandos será

```
Hay 5.000000 vacas en el pastizal /n
```

En la tabla 7.2 se mencionan otros comandos de formato especiales. El tabulador, `\t` es especialmente útil para crear tablas en las que todas las líneas ajusten limpiamente.

Tabla 7.2 Comandos de formato especial

Comando de formato	Acción resultante
<code>\n</code>	salto de línea (linefeed)
<code>\r</code>	regreso de carro (similar a linefeed)
<code>\t</code>	tabulador
<code>\b</code>	retroceder un espacio (backspace)

Puede controlar aún más cómo se despliegan las variables al usar los optativos **width field** y **precision field** con el comando `format`. El **width field** controla el número mínimo de caracteres a imprimir. Debe ser un entero decimal positivo. El **precision field** está precedido por un punto (.) y especifica el número de lugares decimales después del punto decimal para tipos exponencial y punto fijo. Por ejemplo, `%8.2f` especifica que el ancho total mínimo disponible para desplegar su resultado es ocho dígitos, dos de los cuales están después del punto decimal. Por tanto, el código

```
voltage = 3.5;
fprintf('El voltaje es %8.2f milivolts \n',voltage);
```

regresa

```
El voltaje es      3.50 milivolts
```

Note el espacio vacío antes del número 3.50. Esto ocurre porque se reservaron seis espacios (ocho en total, dos después del decimal) para la porción del número a la izquierda del punto decimal.

Muchas veces, cuando usa la función `fprintf`, su variable será una matriz, por ejemplo,

```
x = 1:5;
```

MATLAB repetirá la cadena el comando `fprintf` hasta que use todos los valores en la matriz. Por tanto,

```
fprintf('%8.2f \n',x);
```

regresa

```
1.00
2.00
3.00
4.00
5.00
```

Si la variable es una matriz bidimensional, MATLAB usa los valores una columna a la vez, comenzando por la primera columna, luego la segunda columna, etcétera. He aquí un ejemplo más complicado:

```
feet = 1:3;
inches = feet.*12;
```

Combine estas dos matrices:

```
table = [feet;inches]
```

Entonces MATLAB regresa

```
table =
    1 2 3
   12 24 36
```

Ahora se puede usar la función **fprintf** para crear una tabla que sea más fácil de interpretar. Por ejemplo,

```
fprintf('%4.0f %7.2f \n', table)
```

envía la siguiente salida a la ventana de comandos:

```
1 12.00
2 24.00
3 36.00
```

La función **fprintf** puede aceptar un número variable de matrices después de la cadena. Usa todos los valores en cada una de dichas matrices, en orden, antes de moverse hacia la siguiente matriz. Como ejemplo, suponga que se quiere usar las matrices **feet** e **inches** sin combinarlas en la matriz **tabla**. Entonces se podría escribir

```
fprintf('%4.0f %7.2f \n', feet, inches)
1 2.00
3 12.00
24 36.00
```

La función opera a través de los valores de **feet** primero y luego usa los valores en **inches**. Es improbable que esto sea lo que usted realmente quería que hiciera la función (no lo era en este ejemplo), de modo que los valores de salida casi siempre se agrupan en una sola matriz para usar en **fprintf**.

El comando **fprintf** le da considerablemente más control sobre la forma de su salida que los comandos de formato simple de MATLAB. Sin embargo, requiere cierto cuidado y prudencia usarlo.

Sugerencia

Uno de los errores más comunes cometido por los nuevos programadores es olvidar incluir la **f** en la secuencia del marcador de posición. La función **fprintf** no funciona, pero tampoco se envía mensaje de error.

Sugerencia

Si quiere incluir un signo de porcentaje en un enunciado **fprintf**, necesita ingresar **%** dos veces. Si no lo hace, MATLAB interpretará **%** como un marcador de posición para datos. Por ejemplo,

```
fprintf('La tasa de interés es %5.2f %% \n', 5)
```

resulta en

```
La tasa de interés es 5.00 %
```

EJEMPLO 7.2**Caída libre: salida formateada**

Vuelva a hacer el ejemplo 7.1, pero esta vez cree una tabla de resultados en lugar de una gráfica, y use los comandos **disp** y **fprintf** para controlar la apariencia de la salida.

1. Establezca el problema.
Encontrar la distancia recorrida por un objeto en caída libre.
2. Describa las entradas y salidas.

Entrada Valor de g , la aceleración debida a la gravedad, proporcionada por el usuario
Tiempo, t

Salida Distancias calculadas para cada planeta y la Luna

3. Desarrolle un ejemplo a mano.

$$d = \frac{1}{2}gt^2, \text{ de modo que, en la Luna, a 100 segundos,}$$

$$d = \frac{1}{2} \times 1.6 \text{ m/s}^2 \times 100^2 \text{ s}^2$$

$$d = 8000 \text{ m}$$

4. Desarrolle una solución MATLAB.

```
%Ejemplo 7.2
%Caída libre
clear, clc
%Solicite entrada del usuario
g = input('¿Cuál es el valor de aceleración debida
a gravedad?')
start=input('¿Qué tiempo de inicio le gustaría?')
finish = input('¿Qué tiempo final le gustaría?')
incr = input('¿Qué incrementos de tiempo le gustaría
calcular?')
t=start:incr:finish;
%Calcula la distancia
d=1/2*g*t.^2;
%Crea una matriz de los datos de salida
table=[t;d];
%Envía la salida a la ventana de comandos
fprintf('Para una aceleración debida a la gravedad de %5.1f
segundos \n se calcularon los siguientes datos \n', g)
disp('Distancia recorrida en caída libre')
disp('tiempo, s distancia, m')
fprintf('%8.0f %10.2f\n',table)
```

Este archivo-m produce la siguiente interacción en la ventana de comandos:

```
¿Cuál es el valor de aceleración debida a gravedad?1.6
g =
1.6000
```

```

¿Qué tiempo de inicio le gustaría?0
start =
    0

¿Qué tiempo final le gustaría?100
finish =
    100

¿Qué incrementos de tiempo le gustaría calcular?10
incr =
    10

Para una aceleración debida a la gravedad de 1.6 segundos
se calcularon los siguientes datos

Distance Traveled in Free Fall
tiempo, s      distancia, m
    0           0.00
    10          80.00
    20         320.00
    30         720.00
    40        1280.00
    50        2000.00
    60        2880.00
    70        3920.00
    80        5120.00
    90        6480.00
   100        8000.00

```

5. Ponga a prueba la solución.

Compare la solución MATLAB con la solución a mano. Dado que la salida es una tabla, es fácil ver que la distancia recorrida a 100 segundos es 8000 m. Intente usar otros datos como entrada y compare sus resultados con la gráfica producida en el ejemplo 7.1.

Ejercicio de práctica 7.2

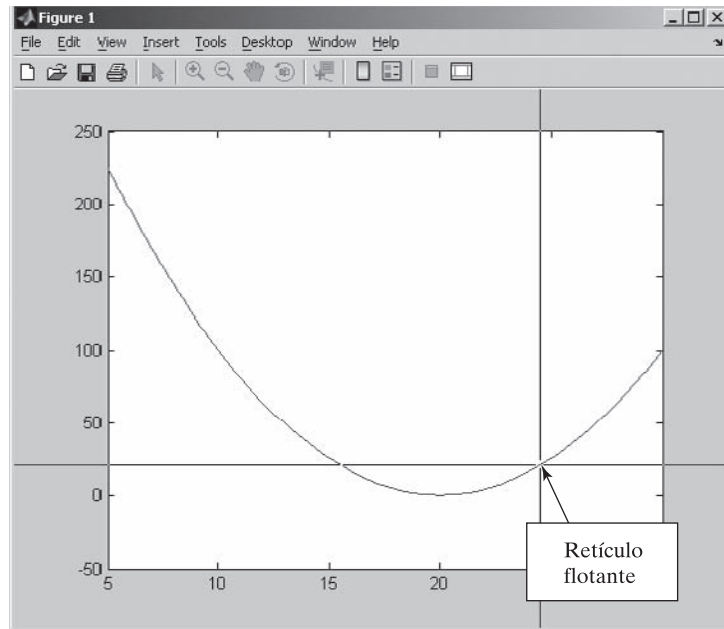
En un archivo-m

1. Use el comando **disp** para crear un título para una tabla que convierte pulgadas a pies.
2. Use el comando **disp** para crear encabezados de columna para su tabla.
3. Cree un vector **inches** desde 0 hasta 120 con un incremento de 10.
Calcule los correspondientes valores de **feet**.
Agrupe el vector **inch** y el vector **feet** en una matriz **table**.
Use el comando **fprintf** para enviar su tabla a la ventana de comandos.

7.3 ENTRADA GRÁFICA

MATLAB ofrece una técnica para ingresar gráficamente pares ordenados de valores x y y . El comando **ginput** permite al usuario seleccionar puntos desde una ventana de figura y convierte los puntos en las coordenadas apropiadas x y y . En el enunciado

```
[x,y] = ginput(n)
```

**Figura 7.3**

La función **ginput** permite al usuario escoger puntos de una gráfica.

MATLAB pide al usuario seleccionar n puntos de la ventana de figura. Si el valor de n no se incluye, como en

```
[x,y] = ginput
```

entonces MATLAB acepta puntos hasta que se ingresa la tecla return.

Esta técnica es útil para escoger puntos de una gráfica. Considere la gráfica en la figura 7.3. La figura se creó al definir x desde 5 hasta 30 y calcular y :

```
x = 5:30;
y = x.^2 - 40.*x + 400;
plot(x,y)
axis([5,30,-50,250])
```

Los valores de eje se definieron de modo que sería más fácil trazarlos.

Una vez que se ejecuta la función **ginput**, como en

```
[a,b] = ginput
```

MATLAB agrega un retículo flotante a la gráfica, como se muestra en la figura 7.3. Luego que este retículo se posiciona a satisfacción del usuario, seleccionar return (enter) envía los valores de las coordenadas x y y al programa:

```
a =
24.4412
b =
19.7368
```

7.4 USO DEL MODO CELDA EN ARCHIVOS-M DE MATLAB

MATLAB 7 tiene una utilidad nueva que permite al usuario dividir los archivos-m en secciones, o celdas (cell), que se pueden ejecutar una a la vez. Esta característica es particularmente

Idea clave: el modo cell (celda) es nuevo a MATLAB 7.

útil conforme usted desarrolla programas MATLAB. El modo cell también permite al usuario crear reportes en varios formatos que muestran los resultados del programa.

Para activar el modo cell, seleccione

Cell → Enable Cell Mode

en la barra de menú de la ventana de edición, como se muestra en la figura 7.4. Una vez habilitado el modo cell, aparece la barra de herramientas cell, como se muestra en la figura 7.5.

Para dividir su programa archivo-m en celdas, puede crear divisores de celda con un doble signo de porcentaje seguido por un espacio. Si quiere nombrar la celda, sólo agregue un nombre en la misma línea del divisor de celda:

%% Cell Name

Idea clave: el modo cell (celda) le permite ejecutar porciones del código incrementalmente.

Una vez que los divisores de celda están en su lugar, si mueve el cursor a alguna parte adentro de la celda, toda la celda se torna amarillo pálido. Por ejemplo, en la figura 7.5, las primeras tres líneas del programa archivo-m constituyen la primera celda. Ahora se pueden usar los iconos de evaluación en la barra de herramientas de celda para evaluar una sola sección, evaluar la sección actual y moverse a la siguiente sección, o evaluar toda la fila. También en la barra de herramientas de celda hay un icono que menciona todos los títulos de celda en el archivo-m, como se muestra en la figura 7.6. La tabla 7.3 muestra los iconos disponibles en la barra de herramientas de celda, junto con sus funciones.

Figura 7.4
Puede acceder al modo celda desde la barra de menú en la ventana de edición.

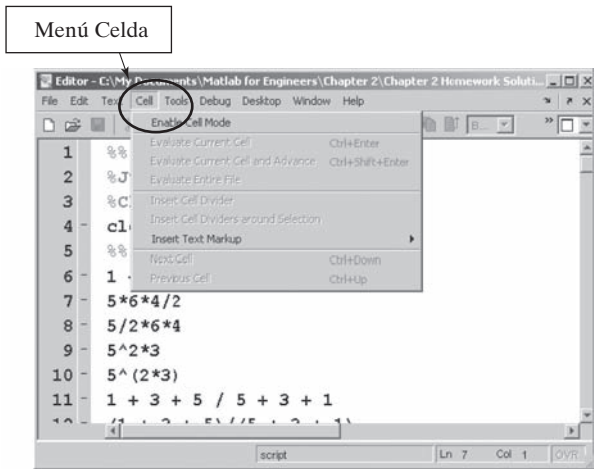
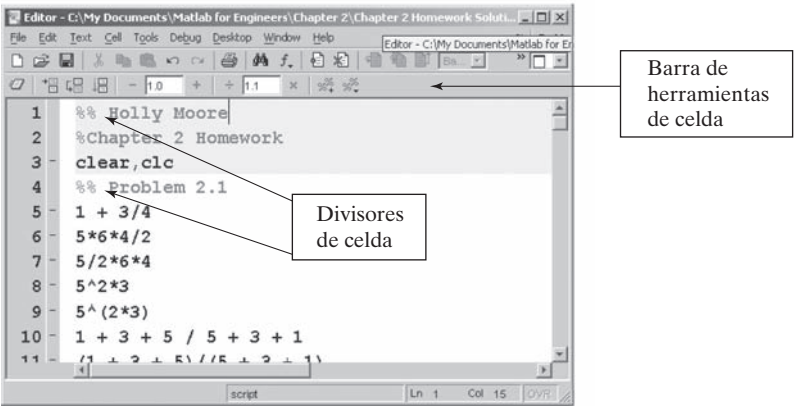


Figura 7.5
La barra de herramientas de celda permite al usuario ejecutar una celda o sección a la vez.



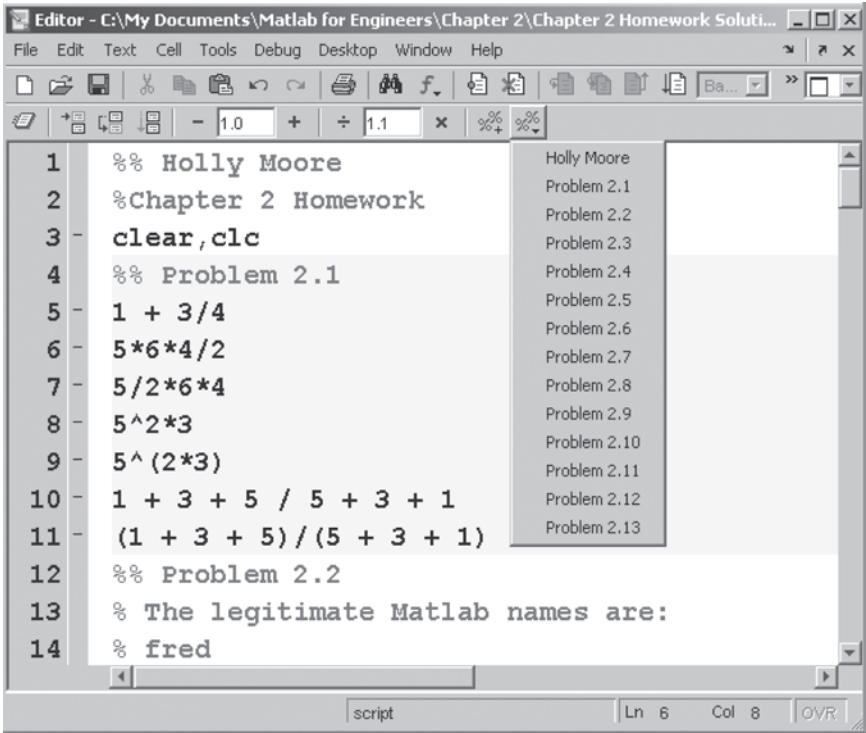


Figura 7.6
El icono Show Cell Titles (mostrar títulos de celda) menciona todas las celdas en el archivo-m.

Tabla 7.3 Barra de herramientas de celda

	evalúa celda actual
	evalúa celda y avanza
	evalúa toda la fila
	muestra títulos de celda
	guarda y publica a HTML

La figura 7.6 muestra las primeras 14 líneas de un archivo-m escrito para resolver algunos problemas de tarea. Al dividir el programa en celdas fue posible trabajar en cada problema por separado. Asegúrese de guardar cualquier archivo-m que desarrolle de esta forma al seleccionar **Save** o **Save As** del menú archivo:

File → Save o

File → Save As

La razón para usar estos comandos es que el programa no se guarda automáticamente cada vez que lo corre.

Dividir en celdas un archivo-m de tarea ofrece una gran ventaja a quien deba calificar el ensayo. Al usar la función **evaluar celda y avanzar**, el calificador puede pasar un problema a la vez a través del programa.

La barra de herramientas de celda también permite al usuario publicar un programa archivo-m a un archivo HTML. MATLAB corre el programa y crea un reporte que muestra el código en cada celda, así como los resultados del cálculo que se enviaron a la ventana de

celda: sección de código MATLAB ubicado entre divisores de celda (%%)

Idea clave: el modo celda le permite crear reportes en HTML, Word y PowerPoint.

comandos. Cualquier figura creada también se incluye en el reporte. La primera porción del reporte creado a partir del archivo-m de la figura 7.6 se muestra en la figura 7.7. Si prefiere un reporte en un formato diferente, como Word o PowerPoint, puede usar la opción de barra de menú

File → Publish To

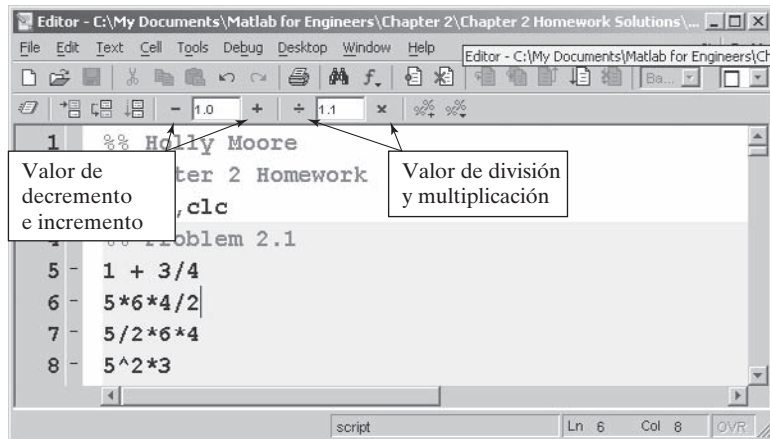
para enviar los resultados en su elección de muchos formatos diferentes.

Finalmente, la barra de herramientas de celda incluye un conjunto de herramientas de manipulación de valor, como se muestra en la figura 7.8.



Figura 7.7

Reporte HTML creado a partir de un archivo-m MATLAB.

**Figura 7.8**

Las herramientas de manipulación de valor permiten al usuario experimentar con diferentes valores en los cálculos.

Cualquier número que esté más cerca del cursor (en la figura 7.8 es el número 2) se puede ajustar por el factor que se muestra en la barra de herramientas al seleccionar el icono apropiado ($-$, $+$, $\div$ o $\times$). Cuando esta característica se usa en combinación con la herramienta **evaluar celda**, puede repetir un conjunto de cálculos muchas veces mientras ajusta fácilmente una variable de interés.

EJEMPLO 7.3**Ajuste interactivo de parámetros**

Sobre la base de un cálculo de equilibrio de energía, usted sabe que el cambio en entalpía de una muestra de 1 kmol (29 kg) de aire que va del estado 1 al estado 2 es 8900 kJ. Usted quisiera saber la temperatura final, pero la ecuación que relaciona el cambio en entalpía a temperatura, a saber,

$$\Delta h = \int_1^2 C_p dT$$

donde

$$C_p = a + bT + cT^2 + dT^3$$

es muy complicada de resolver para la temperatura final. Sin embargo, al usar técnicas aprendida en cálculo, se encuentra que

$$\Delta h = a(T_2 - T_1) + \frac{b}{2}(T_2^2 - T_1^2) + \frac{c}{3}(T_2^3 - T_1^3) + \frac{d}{4}(T_2^4 - T_1^4)$$

Si se conoce la temperatura inicial (T_1) y los valores de a , b , c y d , se pueden adivinar valores para la temperatura final (T_2) hasta obtener el valor correcto de Δh . La habilidad interactiva para modificar valores de variable en el modo celda hace sencilla la resolución de este problema.

1. Establezca el problema.

Encontrar la temperatura final del aire cuando usted conoce la temperatura inicial y el cambio en energía interna.

2. Describa las entradas y salidas.

Entrada Usados en la ecuación para C_p , los siguientes valores de a , b , c y d le darán un valor de capacidad calorífica en kJ/kmol K:

$$a = 28.90$$

$$b = 0.1967 \times 10^{-2}$$

$$c = 0.4802 \times 10^{-5}$$

$$d = -1.966 \times 10^{-9}$$

$$\Delta h = 8900 \text{ kJ}$$

$$T_1 = 300 \text{ K}$$

Salida Para cada valor supuesto de la temperatura final, en la pantalla se debe imprimir una estimación de Δh

3. Desarrolle un ejemplo a mano.

Si supone una temperatura final de 400 K, entonces

$$\Delta h = a(T_2 - T_1) + \frac{b}{2}(T_2^2 - T_1^2) + \frac{c}{3}(T_2^3 - T_1^3) + \frac{d}{4}(T_2^4 - T_1^4)$$

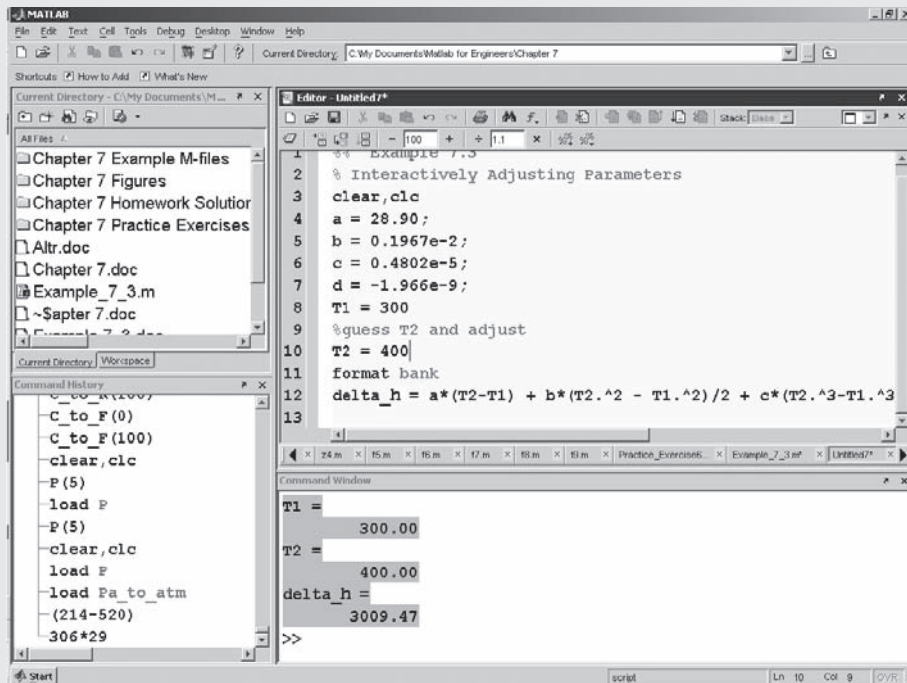
$$\begin{aligned} \Delta h = & 28.9(400 - 300) + \frac{0.1967 \cdot 10^{-2}}{2}(400^2 - 300^2) + \frac{0.4802 \cdot 10^{-5}}{3} \\ & \times (400^3 - 300^3) + \dots \frac{-1.966 \cdot 10^{-9}}{4}(400^4 - 300^4) \end{aligned}$$

que produce

$$\Delta h = 3009.47$$

4. Desarrolle una solución MATLAB.

```
%% Ejemplo 7.3
% Ajuste interactivo de parámetros
clear,clc
a = 28.90;
b = 0.1967e-2;
c = 0.4802e-5;
d = -1.966e-9;
T1 = 300
%% suponga T2 y ajuste
T2 = 400
format bank
delta_h = a*(T2-T1) + b*(T2.^2 - T1.^2)/2 + c*(T2.^3-
T1.^3)/3 + d*(T2.^4-T1.^4)/4
```

**Figura 7.9**

La suposición original le da una idea de cuán alejado está de la respuesta final.

Corra el programa una vez y MATLAB regresa

```
T1 = 300.00
T2 = 400.00
delta_h = 3009.47
```

Ahora coloque el cursor cerca del enunciado **T2 = 400**, como se muestra en la figura 7.9. (En este ejemplo, la ventana de edición se acopló con el escritorio MATLAB.) Al seleccionar el icono Increment Value, con el valor establecido a 100, se pueden intentar rápidamente muchas temperaturas diferentes. (Véase la figura 7.10.) Una vez que cierra, puede cambiar el incremento y el cero en la respuesta.

Un valor T_2 de 592 K dio un valor Δh calculado de 8927, que está bastante cerca de la meta. Se podría acercar más si creyera que la precisión añadida estaba justificada.

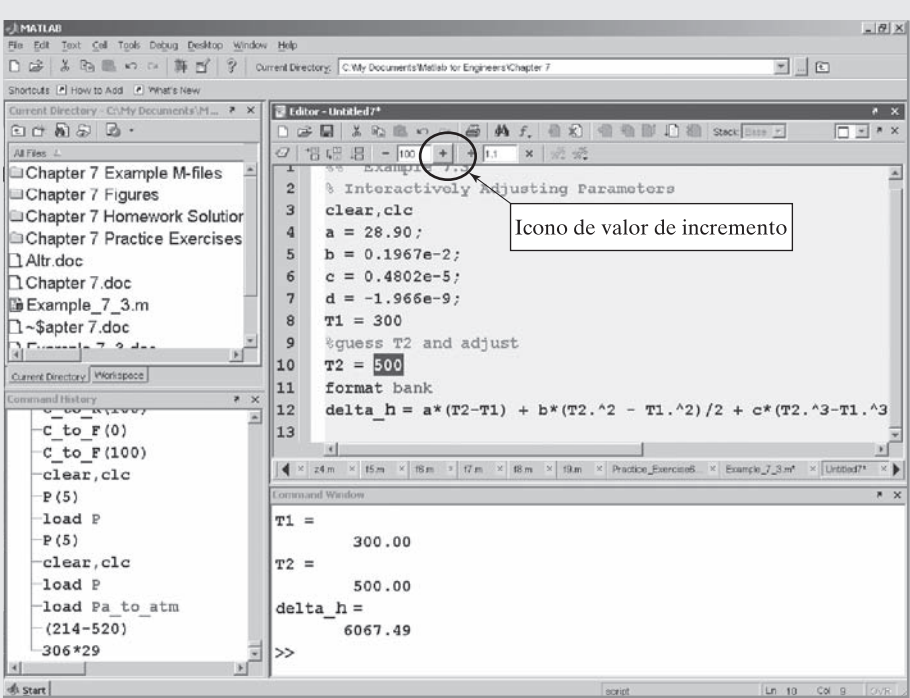
5. Ponga a prueba la solución.

Sustituya el valor calculado de T_2 en la ecuación original y verifique los resultados con una calculadora:

$$\begin{aligned}\Delta h &= 28.9(592 - 300) + \frac{0.1967 \cdot 10^{-2}}{2}(592^2 - 300^2) + \\ &\quad + \frac{0.4802 \cdot 10^{-5}}{3}(592^3 - 300^3) + \frac{-1.966 \cdot 10^{-9}}{4}(592^4 - 300^4) \\ \Delta h &= 8927.46\end{aligned}$$

Figura 7.10

Ajuste el valor más cercano al cursor al seleccionar uno de los iconos Increment/Decrement y ajustar el tamaño de paso que se muestra en la barra de herramientas del modo celda.



7.5 LECTURA Y ESCRITURA DE DATOS DESDE ARCHIVOS

Idea clave: MATLAB puede importar datos de archivos que usan diversos formatos.

Los datos se almacenan en muchos formatos diferentes, dependiendo de los dispositivos y programas que crean los datos y de la aplicación. Por ejemplo, el sonido se puede almacenar en un archivo .wav, y una imagen se puede almacenar en un archivo .jpg. Muchas aplicaciones almacenan datos en hojas de cálculo Excel (archivos .xls). El más genérico de estos archivos es el archivo ASCII, que usualmente se almacena como un archivo .dat o .txt. Quizá usted quiera importar estos datos en MATLAB para analizar en un programa MATLAB, o tal vez quiera guardar sus datos en uno de estos formatos para hacer el archivo más fácil de exportar a otra aplicación.

7.5.1 Importación de datos

Importa Wizard (asistente de importación)

Si usted selecciona un archivo de datos del directorio actual y hace doble clic en el nombre del archivo, se lanza el Import Wizard (asistente de importación). El Import Wizard determina qué tipo de datos hay en el archivo y sugiere formas de representar los datos en MATLAB. La tabla 7.4 es una lista de algunos de los tipos de datos reconocidos por MATLAB. MATLAB no soporta todos los posibles formatos de datos. Puede encontrar una lista completa al escribir

`doc fileformats`

en la ventana de comandos.

Tabla 7.4 Tipos de archivo de datos soportados por MATLAB

Tipo de archivo	Extensión	Observación
Texto	.mat	área de trabajo MATLAB
	.dat	datos ASCII
	.txt	datos ASCII
Otros formatos comunes de datos científicos	.cdf	formato de datos común
	.fits	datos en sistema flexible de transporte de imagen
	.hdf	formato de datos jerárquicos
Datos de hoja de cálculo	.xls	hoja de cálculo Excel
	.wk1	Lotus 123
	.tiff	formato de archivo de imagen etiquetado
Datos de imagen	.bmp	mapa de bits
	.jpeg o jpg	grupo experto fotográfico unido
	.gif	formato de intercambio gráfico
Datos de audio	.au	audio
	.wav	archivo wave Microsoft
Película	.avi	archivo intercalado audio/video

El Import Wizard se puede usar para archivos ASCII simples y para archivos de hoja de cálculo Excel. También puede lanzar el Import Wizard desde la línea de comando, con la función **uiimport**:

```
uiimport(' nombre de archivo.extensión ')
```

Por ejemplo, para importar el archivo de sonido **decision.wav**, escriba

```
uiimport(' decision.wav ')
```

Entonces se abre el Importa Wizard, como se muestra en la figura 7.11.

Cualquier técnica para lanzar el Importa Wizard requiere una interacción con el usuario (a través del Wizard). Si quiere cargar un archivo de datos desde un programa MATLAB, necesitará un enfoque diferente.

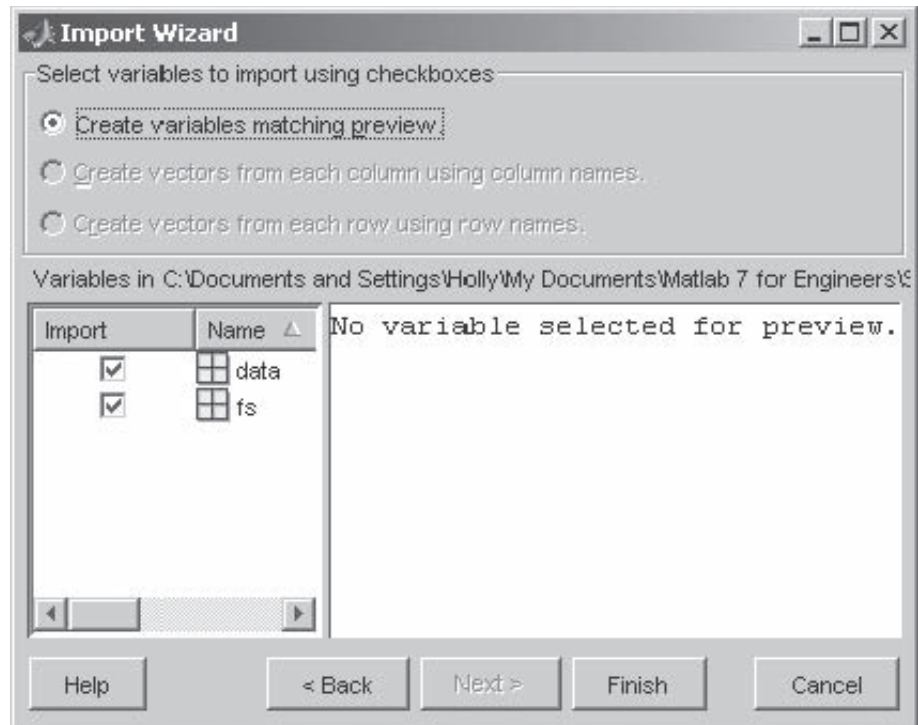
Comandos de importación

Puede evitar las interacciones del Wizard con una de las funciones que están especialmente diseñadas para leer cada uno de los formatos de archivo soportados. Por ejemplo, para leer un archivo .wav, use la función **wavread**:

```
[data,fs]=wavread('decision.wav')
```

Claramente, necesita entender qué tipo de datos esperar, de modo que pueda nombrar apropiadamente las variables creadas. Puede encontrar una lista de funciones de importación al escribir

```
doc fileformats
```

**Figura 7.11**

El Import Wizard se lanza cuando se ejecuta el comando **uiimport**.

EJEMPLO 7.4

2001: una odisea espacial: archivos de sonido

Uno de los personajes más memorables de la película *2001: Odisea del espacio* fue la computadora Hal. Los sonidos del diálogo de Hal en la película han sido populares durante años entre los programadores de computadoras y los ingenieros que usan computadoras. Usted puede encontrar archivos .wav de algunos de los diálogos de Hal en <http://www.palantir.net/2001/> y en otros websites de cultura popular. Inserte los comentarios de Hal en un programa MATLAB. (Necesitará la función **sound**; consulte el tutorial **help** para detalles acerca de su uso.)

1. Establezca el problema.
Cargar archivos de sonido en un programa MATLAB y reproducíalos en momentos apropiados.
2. Describa las entradas y salidas.

Entrada Archivos de sonidos descargados de Internet. Para este ejemplo, se supondrá que descargó los siguientes tres archivos:

dave.wav
error.wav
sure.wav

Salida Reproducir los archivos de sonido dentro de un programa MATLAB

3. Desarrolle un ejemplo a mano.
Aunque trabajar un ejemplo a mano no es apropiado para este problema, puede escuchar los archivos de sonido de Internet antes de insertarlos en el programa.

4. Desarrolle una solución MATLAB.

Descargue los archivos de sonido y guárdelos en el directorio actual antes de correr el siguiente programa:

```
%% Ejemplo 7.4
% Archivos de sonido
%% Primer clip
[dave,fs_dave]=wavread('dave.wav');
disp('Presione enter una vez que el clip de sonido deje de tocar')
sound(dave,fs_dave)
pause
%% Segundo clip
[error,fs_error]=wavread('error.wav');
disp('Presione enter una vez que el clip de sonido deje de tocar')
sound(error,fs_error)
pause
%% Tercer clip
[sure,fs_sure]=wavread('sure.wav');
disp('Presione enter una vez que el clip de sonido deje de tocar')
sound(sure,fs_sure)
pause
disp('Ése fue el último clip')
```

5. Ponga prueba la solución.

Existen muchos archivos de audio disponibles para descargar de Internet. Muchos son tan simples como éstos, pero algunos son piezas musicales completas. Navegue en Internet e inserte un “byte sonoro” en otro programa MATLAB, como un mensaje de error para sus usuarios. Algunos de los favoritos del autor son *Star Trek* (intente <http://services.tos.net/sounds/sound.html#tos>) y *Los Simpsons*.

7.5.2 Exportación de datos

La forma más sencilla de encontrar la función adecuada para escribir un archivo es usar el tutorial **help** para encontrar la función correcta para leerla y luego seguir las ligas hacia la función **write**. Por ejemplo, para leer un archivo de hoja de cálculo Excel (.xls), se usaría **xlsread**:

```
xlsread('nombre de archivo.xls')
```

Al final de la página tutorial se hace referencia a la función correcta para escribir un archivo Excel, a saber,

```
xlswrite('nombre de archivo.xls', M)
```

donde **M** es el arreglo que quiere almacenar en la hoja de cálculo Excel.

RESUMEN

MATLAB proporciona funciones que permiten al usuario interactuar con un programa archivo-m y que permiten al programador controlar la salida a la ventana de comandos.

La función **input** pausa el programa y envía una incitación determinada por el programador a la ventana de comandos. Una vez que el usuario ingresó un valor o valores y oprime la tecla return, continúa la ejecución del programa.

El comando **display** (**disp**) permite al programador desplegar los contenidos de una cadena o una matriz en la ventana de comandos. Aunque el comando **disp** es adecuado para muchas tareas de despliegue, el comando **fprintf** da al programador considerablemente más control sobre la forma en que los resultados se despliegan en la ventana de comandos. Ello permite al programador combinar texto y resultados calculados en la misma línea y especificar el número de formato usado.

Para aplicaciones en las que se requiere entrada gráfica, el comando **ginput** permite al usuario proporcionar entrada a un programa al seleccionar puntos de una ventana de gráficos.

El modo celda permite al programador agrupar código de archivo-m en secciones y correr cada sección de manera individual. La herramienta **publish to HTML** crea un reporte que contiene tanto el código del archivo-m y los resultados, así como cualesquiera figuras generadas cuando se ejecutó el programa. Los iconos Incremento y Decremento en la barra de herramientas de celda permiten al usuario cambiar automáticamente el valor de un parámetro cada vez que el código se ejecuta, lo que hace sencillo probar el resultado de cambiar una variable.

MATLAB incluye funciones que permiten al usuario importar y exportar datos en algunos formatos de archivo populares. Una lista completa de dichos formatos está disponible en el tutorial **help** en la página File Formats (doc fileformats).

RESUMEN MATLAB

El siguiente resumen MATLAB menciona todos los caracteres, comandos y funciones especiales que se definieron en este capítulo:

Caracteres especiales	
'	comienza y termina una cadena
%	marcador de posición usado en el comando fprintf
%f	notación punto fijo, o decimal
%e	notación exponencial
%g	notación o punto fijo o exponencial
%s	notación cadena
%%	divisor de celda
\n	salto de línea (linefeed)
\r	regreso de carro (similar a linefeed)
\t	tabulador
\b	retroceder un espacio (backspace)

Comandos y funciones	
disp	despliega una cadena o una matriz en la ventana de comandos
fprintf	controla el despliegue de ventana de comandos
ginput	permite al usuario elegir valores de una gráfica
input	permite al usuario ingresar valores
num2str	cambia un número a una cadena
pause	pausa el programa
sound	reproduce datos MATLAB a través de las bocinas
uiimport	lanza el Importa Wizard
wavread	lee archivos wave
xlsimport	importa archivos de datos Excel
xlswrite	exporta datos como un archivo Excel

ancho de campo
arreglo carácter
cadena

campo precisión
celda
modo celda

salida formateada

TÉRMINOS CLAVE

PROBLEMAS

Función input

- 7.1 Cree un archivo-m que conmine al usuario a ingresar un valor de x y luego calcule el valor de $\sin(x)$.
- 7.2 Cree un archivo-m que conmine al usuario a ingresar una matriz y luego use la función **max** para determinar el valor ingresado más grande. Use la siguiente matriz para probar su programa:

[1,5,3,8,9,22]

- 7.3 El volumen de un cono es

$$V = \frac{1}{3} \times \text{área_de_la_base} \times \text{altura}$$

Conmine al usuario a ingresar el área de la base y la altura del cono (figura P7.3). Calcule el volumen del cono.

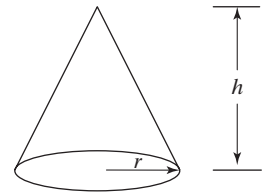


Figura P7.3
Volumen de un cono.

Función disp

- 7.4 Uno de los primeros programas de cómputo que muchos estudiantes escriben se llama “Hola, mundo”. Lo único que hace el programa es imprimir este mensaje en la pantalla de la computadora. Escriba un programa “Hola, mundo” en un archivo-m con la función **disp**.
- 7.5 Use dos enunciados **input** separados para conminar al usuario a ingresar su nombre y apellidos. Use la función **disp** para desplegar dichos nombres en una línea. (Necesitará combinar los nombres y algunos espacios en un arreglo.)
- 7.6 Conmine al usuario a ingresar su edad. Luego use la función **disp** para reportar la edad de vuelta a la ventana de comandos. Si, por ejemplo, el usuario ingresa 5 cuando se le solicita la edad, su despliegue debe leerse

Su edad es 5

Esta salida requiere combinar tanto datos carácter (una cadena) como datos numéricos en la función **disp**, lo que se puede lograr al usar la función **num2str**.

- 7.7 Conmine al usuario a ingresar un arreglo de números. Use la función **length** para determinar cuántos valores ingresó y use la función **disp** para reportar sus resultados a la ventana de comandos.

fprintf

- 7.8 Repita el problema 7.7 y use **fprintf** para reportar sus resultados.
- 7.9 Use **fprintf** para crear las tablas de multiplicación de 1 a 13 para el número 6. Su tabla se debe ver como esto

1 por 6 es 6
2 por 6 es 12
3 por 6 es 18
⋮

- 7.10** Antes de que las calculadoras fueran fácilmente asequibles (alrededor de 1974), los estudiantes usaban tablas para determinar los valores de las funciones matemáticas como seno, coseno y log. Cree una de tales tablas para seno, con los siguientes pasos:
- Cree un vector de valores ángulo desde 0 hasta 2π en incrementos de $\pi/10$.
 - Calcule el seno de cada uno de los ángulos y agrupe sus resultados en una tabla que incluya el ángulo y el seno.
 - Use **disp** para crear un título para la tabla y un segundo comando **disp** para crear encabezados de columna.
 - Use la función **fprintf** para desplegar los números. Despliegue sólo dos valores después del punto decimal.
- 7.11** Las dimensiones muy pequeñas, las que están a escala atómica, con frecuencia se miden en angstroms. El símbolo para un angstrom es Å y corresponde a una longitud de 10^{-10} metros. Cree una tabla de conversión de pulgadas a angstroms del modo siguiente, para valores de pulgadas desde 1 hasta 10:
- Use **disp** para crear un título y encabezados de columna.
 - Use **fprintf** para desplegar la información numérica.
 - Puesto que la longitud representada en angstroms es demasiado grande, represente su resultado en notación científica y muestre dos valores después del punto decimal. Esto corresponde a tres cifras significativas (una antes y dos después del punto decimal).
- 7.12** Use su buscador favorito de Internet y navegue la red para identificar conversiones monetarias recientes para libras esterlinas británicas, yen japonés y el euro europeo a dólares estadounidenses. Use las tablas de conversión para crear las siguientes tablas (use los comandos **disp** y **fprintf** en su solución, que debe incluir un título, etiquetas de columna y salida formateada):
- (a) Genere una tabla de conversiones de yen a dólar. Comience la columna yen en 5 e incremente por 5 yen. Imprima 25 líneas en la tabla.
 - (b) Genere una tabla de conversiones de euros a dólares. Comience la columna euro en 1 euro e incremente por 2 euros. Imprima 30 líneas en la tabla.
 - (c) Genere una tabla con cuatro columnas. La primera debe contener dólares, la segunda debe contener el número equivalente de euros, la tercera el número equivalente de libras y la cuarta el número equivalente de yen. Haga que la columna dólar varíe de 1 a 10.

Problemas que combinan los comandos input, disp y fprintf

- 7.13** Este problema requiere que usted genere tablas de conversión de temperatura. Use las siguientes ecuaciones, que describen las relaciones entre temperaturas en grados Fahrenheit (T_F), grados Celsius (T_C), grados Kelvin (T_K) y grados Rankine (T_R), respectivamente:

$$T_F = T_R - 459.67 \text{ }^\circ\text{R}$$

$$T_F = \frac{9}{5}T_C + 32 \text{ }^\circ\text{F}$$

$$T_R = \frac{9}{5}T_K$$

¡Necesitará reordenar estas expresiones para resolver algunos de los problemas!

- (a) Genere una tabla de conversiones de Fahrenheit a Kelvin para valores desde 0 °F hasta 200 °F. Permita que el usuario ingrese los incrementos en grados F entre líneas. Use **disp** y **fprintf** para crear una tabla con un título, encabezados de columna y espaciado adecuado.

- (b) Genere una tabla de conversiones de Celsius a Rankine. Permite que el usuario ingrese la temperatura inicial y los incrementos entre líneas. Imprima 25 líneas en la tabla. Use **disp** y **fprintf** para crear una tabla con un título, encabezados de columna y espaciamiento apropiado.
- (c) Genere una tabla de conversiones de Celsius a Fahrenheit. Permita que el usuario ingrese la temperatura inicial, el incremento entre líneas y el número de líneas para la tabla. Use **disp** y **fprintf** para crear una tabla con un título, encabezados de columna y espaciamiento apropiado.

7.14 Los ingenieros usan regularmente unidades tanto inglesas como SI (Système International d'Unités). Algunos campos usan principalmente uno u otro, pero muchos combinan los dos sistemas. Por ejemplo, la tasa de entrada de energía a una planta de potencia de vapor a partir de la quema de combustibles fósiles usualmente se mide en Btu/hora. Sin embargo, la electricidad producida por la misma planta, por lo general, se mide en joules/s (watts). En contraste, los motores de automóvil con frecuencia se califican en caballos de fuerza o en pie lb_f/s. He aquí algunos factores de conversión que relacionan estas diferentes mediciones de potencia:

$$1 \text{ kW} = 3412.14 \text{ Btu/h} = 737.56 \text{ ft lb}_f/\text{s}$$

$$1 \text{ hp} = 550 \text{ ft lb}_f/\text{s} = 2544.5 \text{ Btu/h}$$

- (a) Genere una tabla de conversiones de kW a hp. La tabla debe comenzar en 0 kW y terminar en 15 kW. Use la función **input** para permitir al usuario definir el incremento entre entradas de la tabla. Use **disp** y **fprintf** para crear una tabla con un título, encabezados de columna y espaciamiento apropiado.
- (b) Genere una tabla de conversiones de ft lb_f/s a Btu/h. La tabla debe comenzar en 0 kW, pero permitir al usuario definir el incremento entre entradas de tabla y el valor final de la tabla. Use **disp** y **fprintf** para crear una tabla con un título, encabezados de columna y espaciamiento apropiado.
- (c) Genere una tabla que incluya conversiones de kW a Btu/h y ft lb_f/s. Permita al usuario definir el valor inicial de kW, el valor final de kW y el número de entradas en la tabla. Use **disp** y **fprintf** para crear una tabla con un título, encabezados de columna y espaciamiento apropiado.

ginput

7.15 En el tiempo $t = 0$, el motor de un cohete se apaga con el cohete habiendo alcanzado una altitud de 500 metros y elevándose con una rapidez de 125 metros por segundo. En este punto, la gravedad toma el control. La altura del cohete como función del tiempo es

$$h(t) = -\frac{9.8}{2}t^2 + 125t + 500 \text{ para } t > 0$$

Grafique la altura del cohete desde 0 hasta 30 segundos y

- Use la función **ginput** para estimar la altura máxima que el cohete alcanza y el tiempo cuando el cohete golpea el suelo.
- Use el comando **disp** para reportar sus resultados a la ventana de comandos.

7.16 La función **ginput** es útil para escoger distancias de una gráfica. Demuestre esta característica mediante la realización de lo siguiente:

- Cree una gráfica de un círculo mediante la definición de un arreglo de ángulos desde 0 hasta 2π , con un espaciamiento de $\pi/100$.
- Use la función **ginput** para elegir dos puntos en la circunferencia del círculo.

- Use **hold on** para evitar que la figura se refresque y grafique una línea entre los dos puntos que eligió.
- Use los datos de los puntos para calcular la longitud de la línea entre ellos. (*Sugerencia:* use el teorema de Pitágoras en su cálculo.)

Modo celda

- 7.17** Cree un archivo-m que contenga sus soluciones a los problemas de tarea de este capítulo. Use divisores de celda (%%) para dividir su programa en celdas (secciones) y titule cada sección con un número de problema. Corra su programa con la característica **evaluar celda y avanzar** de la barra de herramientas de celda.
- 7.18** Publique su programa y resultados del problema 7.17 en HTML, con la característica **publish to HTML** de la barra de herramientas de celda. Desafortunadamente, dado que la tarea de este capítulo requiere interacción con el usuario, los resultados publicados incluirán errores.